

Advanced NLP Assignment-1 Report

Phanindra Gollapalli | Roll Number: 2024101068

Links:

- **Hugging Face Checkpoints:** <https://huggingface.co/phani4104/Transformer>
- **WandB Project:** https://wandb.ai/phani1729-iiit-hyderabad/ANLP_A1
- C1 Baseline,
- C2 RoPE,
- C3 GQA,
- C4 RMSNorm,
- C5 BLT

1 From-Scratch Architectural Components

1.1 Scaled Dot-Product Attention

Given the query matrix $Q \in \mathbb{R}^{N \times d_k}$, key matrix $K \in \mathbb{R}^{M \times d_k}$, and value matrix $V \in \mathbb{R}^{M \times d_v}$:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} + M_{\text{attn}} \right) V \quad (1)$$

I added a masking tensor (M_{attn}) that places $-\infty$ at blocked positions and 0 at valid ones. This handles both standard key padding and the causal mask needed for the autoregressive decoder.

1.2 Multi-Head Attention (MHA) vs. Grouped-Query Attention (GQA)

For the standard MHA setups (C1, C2, C4, C5), queries, keys, and values are linearly projected into h independent heads:

$$\text{MHA}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O, \quad \text{where } \text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \quad (2)$$

For GQA (C3), I grouped the $h = 8$ query heads into $G = 4$ groups. Every query head still gets its own projection, but the keys and values share a projection within each group:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_{\lfloor i/2 \rfloor}^K, VW_{\lfloor i/2 \rfloor}^V) \quad (3)$$

This essentially cuts the parameter count and cache memory for keys and values in half, while keeping the model almost just as expressive.

1.3 Positional Encodings: Sinusoidal Absolute vs. RoPE

Sinusoidal Encodings (C1, C3, C4, C5) just add absolute position values directly to the input embeddings:

$$PE_{(pos, 2i)} = \sin \left(\frac{pos}{10000^{2i/d_{\text{model}}}} \right), \quad PE_{(pos, 2i+1)} = \cos \left(\frac{pos}{10000^{2i/d_{\text{model}}}} \right) \quad (4)$$

Rotary Position Embeddings / RoPE (C2) Instead of adding absolute positions, they rotate the projected queries and keys in 2D sub-planes:

$$R_{\Theta, m}^d = \text{diag} \left(R_{\theta_{1,m}}, \dots, R_{\theta_{d/2,m}} \right), \quad \tilde{q}_m = R_{\Theta, m}^d (W^Q x_m), \quad \tilde{k}_n = R_{\Theta, n}^d (W^K x_n) \quad (5)$$

With RoPE, the dot product $\langle \tilde{q}_m, \tilde{k}_n \rangle$ naturally captures the relative distance between tokens, rather than relying on absolute positions getting warped as they pass through deep layers.

1.4 Normalization: Pre-LayerNorm vs. RMSNorm

I used Pre-Layer Normalization (Pre-LN) everywhere to keep gradients stable. **LayerNorm** (C1, C2, C3, C5) normalizes both the mean and the variance:

$$\text{LN}(x) = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \odot \gamma + \beta \quad (6)$$

RMSNorm (C4) takes a shortcut by completely skipping the mean-centering step:

$$\text{RMSNorm}(x) = \frac{x}{\text{RMS}(x)} \odot \gamma, \quad \text{where } \text{RMS}(x) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon} \quad (7)$$

Dropping the mean calculation simplifies the math and makes the backward pass noticeably faster on the GPU.

1.5 Byte Latent Transformer (C5)

For C5, We completely ditch the BPE tokenizer to see if the model could learn directly from raw UTF-8 bytes using a 3-stage Byte Latent Transformer (BLT) pipeline:

1. **Source Local Patch Encoder:** The raw cipher bits are packed into bytes, embedded, and chopped into fixed patches of size $P = 16$. A local self-attention layer refines them, and then I used masked pooling to compress each patch into a single latent vector. The 4-layer global encoder only looks at these patch latents ($N_{\text{src}} = S/16$).
2. **Shifted Target Latent Path:** Target bytes are also patched and compressed. To stop the model from cheating (autoregressive causality), I prepended a learned `global_bos_patch` and shifted everything to the right by 1 patch:

$$H_{\text{global_dec_in}} = [\text{global_bos_patch}, \text{target_patch_latents}[:, :, -1]] \quad (8)$$

The global decoder then processes this short sequence of patches.

3. **Local Byte Autoregressive Decoder:** Finally, the local byte decoder takes target byte embeddings and cross-attends to the global patch states using a very strict causal mask:

$$M_{\text{cross}}(t, j) = (j \leq \lfloor t/P \rfloor) \quad (9)$$

This guarantees that predicting a byte in patch p only uses global information from patches that have already been fully processed.

2 Experimental Setup

2.1 Dataset and Preprocessing

I used the provided Brown corpus dataset, which contains 5000 cipher-plaintext sentence pairs. I chopped these into 64-byte plaintext chunks (which match up with 512-bit cipher chunks). So it became 49k cipher-plaintext sequences. I did chunking on the dataset because I noticed the model was struggling to predict entire long sentences on a smaller set (only 5,000 pairs). Chunking increased the number of training examples and made it easier for the model to learn the direct mapping between 64-byte plaintext and 512-bit cipher chunks. I used a strict 80/10/10 train/val/test split and set the random seed to 42 so the results are perfectly reproducible.

For runs C1–C4, I trained a standard Byte-Pair Encoding (BPE) tokenizer from scratch with vocab size $V = 760$. For C5, the model just ingested raw byte values ($V = 260$, reserving IDs 0–3 for PAD, BOS, EOS, UNK, and 4–259 for the actual byte values 0–255).

2.2 Controlled Hyperparameters

To make sure the ablation study was actually valid, I froze the core architecture and training settings across all five configurations:

- **Architecture:** 4 encoder layers, 4 decoder layers; $d_{\text{model}} = 256$, $d_{\text{ff}} = 1024$, $h = 8$ heads, Dropout = 0.1.
- **Optimization:** AdamW optimizer ($\beta_1 = 0.9$, $\beta_2 = 0.98$, weight decay = 10^{-2}), learning rate = 3×10^{-4} , gradient clipping = 1.0, batch size = 64.
- **Evaluation:** Greedy decoding on the 4,988 held-out test sequences. (this is after chopping the dataset) I tracked Bit Accuracy, Sequence Accuracy, Levenshtein Distance, BLEU, and ROUGE-L.

3 Empirical Results and Comparative Analysis

Table 1: Controlled Ablation Study Results (C1–C5) evaluated on the held-out test set.

Metric	C1 (Base)	C2 (RoPE)	C3 (GQA)	C4 (RMSNorm)	C5 (BLT)
Positional Enc.	Sinusoidal	RoPE	Sinusoidal	Sinusoidal	Sinusoidal
Attention Type	MHA	MHA	GQA (4 grp)	MHA	MHA
Normalization	LayerNorm	LayerNorm	LayerNorm	RMSNorm	LayerNorm
Tokenization	Subword BPE	Subword BPE	Subword BPE	Subword BPE	Token-Free BLT
Parameters	7,952,120	7,952,120	7,162,616	7,949,560	9,873,156
Peak GPU Memory	1212.15 MB	1212.06 MB	1202.25 MB	1091.14 MB	608.31 MB
Total Train Time	1693.10 s	1867.87 s	1630.75 s	1604.79 s	1306.09 s
Best Val Loss	0.0201	0.0083	0.0211	0.0168	0.2418
Bit Accuracy (%)	99.01%	99.60%	99.00%	99.01%	84.15%
Sequence Acc. (%)	89.78%	95.06%	89.34%	90.15%	3.48%
Levenshtein Dist.	0.185	0.083	0.206	0.185	18.338
BLEU Score	0.9742	0.9886	0.9733	0.9753	N/A
ROUGE-L F1	0.9878	0.9930	0.9880	0.9880	N/A

3.1 Impact of Rotary Position Embeddings (C1 vs. C2)

Swapping the standard sinusoidal encodings for RoPE was easily the best change in the whole assignment.

- The sequence exact match jumped from 89.78% to a massive 95.06%, and the edit distance was from (0.185 $\rightarrow$ 0.083).
- **Reason:** Decrypting ciphers means the model desperately needs to track exact relative bit offsets. RoPE preserves relative position inner products $\langle q_m, k_n \rangle = g(m - n)$ perfectly across all attention heads, avoiding the degradation we get with sinusoidal embeddings in deeper layers.

3.2 Impact of Grouped-Query Attention (C1 vs. C3)

Grouping the 8 query heads into 4 key-value groups dropped the parameter count by about 10%.

- Training time got a nice little $\sim 4\%$ bump in speed.
- The crazy part is that the accuracy metrics barely budged at all (89.34% vs 89.78% sequence accuracy).
- GQA works exactly as advertised. It gives us a lighter, more memory-efficient model for practically free.

3.3 Impact of RMSNorm (C1 vs. C4)

Dropping the mean-centering step in the normalization layers resulted in:

- A 10% reduction in peak GPU memory (1091 MB down from 1212 MB).
- A 5.2% faster training time.
- It slightly improved the validation loss and sequence accuracy compared to the baseline.
- RMSNorm simplifies the backward pass computation on the GPU while keeping everything perfectly stable.

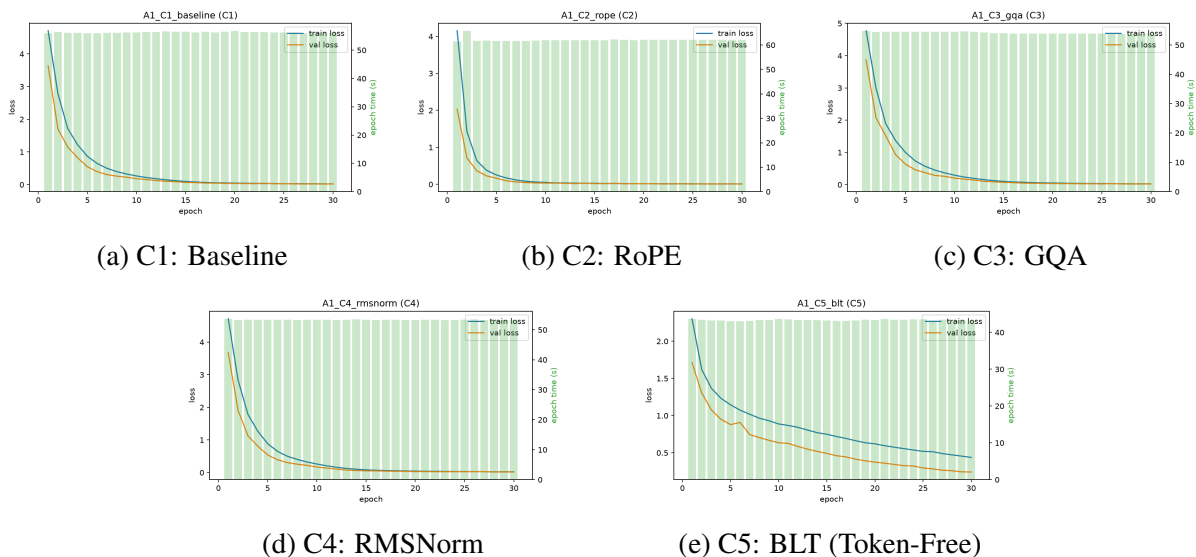


Figure 1: **Training and Validation Loss Curves** across all 30 epochs.

4 Byte Latent Transformer (C5)

4.1 Advantages of BLT: Memory and Speed

Because standard self-attention scales quadratically with sequence length, compressing 64 bytes into just 4 patches ($P = 16$) basically decreases a lot of memory requirement.

- C5 used only 608.31 MB of VRAM, compared to 1212.15 MB for C1.
- **23% Faster Training:** C5 finished 30 epochs in just 1306.09 s.

4.2 The Catch: Compounding Errors and Accuracy Drops

Despite the massive compute savings, C5's accuracy tanked compared to the baseline. It got 84.15% bit accuracy, but a brutal 3.48% exact sequence match. When you look at the actual text it generates, you can see exactly what's going wrong:

Ref: 'Credits adapted from the Loose liner notes'

Pred: 'Cradisted topearfrom the Loose liner tones'

Ref: 'In January the embassy arrived at Beijing and was received by th'

Pred: 'In January the embassy reviduat Minge a was fis received by th'

The model is clearly learning English syntax and figuring out the cipher patterns, but it completely fails at exact sequence matching for two main reasons (I think) :

1. **Compounding Error over 64 Steps:** The C1 model only has to decode about 18 BPE tokens per chunk. C5 has to decode 64 individual bytes in a row perfectly. Even if it gets 98% accuracy on every single step, 0.98^{64} gives you terrible odds of getting the whole sequence right. Just one wrong character ruins the sequence match metric.
2. **Fixed 16-Byte Patching is Clunky:** Meta’s original paper uses dynamic patching to cleanly split text at word boundaries. Because I only used a fixed $P = 16$ patch size, the 16-byte boundaries blindly bisect words and morphemes across patch cuts. Compressing these disjoint character fragments into one vector makes it much harder for the local decoder to reconstruct fluent, error-free words from scratch.